

Reverse, Difference & Count of 2s

Company: Walmart (SDE 2)

Round: Online Assessment (OA) - Coding Round, Question 1

Year: 2024

Difficulty: Medium

Topic: Math, Number Manipulation, Prime Factorization

Source: [LeetCode Discuss – Walmart SDE 2 OA Experience \(Sept 2024\)](#)

Problem Description

You are given a non-negative integer n . Reverse the digits of n to obtain a second number, then compute the absolute difference between the original number and its reversed form. Finally, determine how many times 2 divides that absolute difference — that is, the exponent of 2 in its prime factorization.

Input Format

The input consists of a single line containing one non-negative integer n .

Output Format

Print a single integer — the number of times 2 occurs as a factor of $|n - \text{reverse}(n)|$.

Constraints

$$0 \leq n \leq 10^9$$

Sample Input / Output

Example 1

Input:

123

Output:

1

Example 2

Input:

48

Output:

2

Example 3 (Palindrome edge case)

Input:

121

Output:

0

Explanation

In Example 1, reversing 123 gives 321. The absolute difference is $|123 - 321| = 198$. Factoring 198 gives $2 \times 3^2 \times 11$, so 2 appears exactly once — the answer is 1.

In Example 2, reversing 48 gives 84. The absolute difference is $|48 - 84| = 36$. Factoring 36 gives $2^2 \times 3^2$, so 2 appears twice — the answer is 2.

In Example 3, 121 is a palindrome, so reversing it gives 121 again. The absolute difference is 0. Since 0 has no meaningful prime factorization, this is treated as a special case with an answer of 0.

Approach to Solve This Problem:

This problem breaks down into two simple stages: reversing the digits of n , and then counting how many times 2 divides the resulting absolute difference. Both stages rely only on basic arithmetic, so the entire solution runs in $O(\log n)$ time using a constant number of variables.

The one case that needs explicit handling is when n is a palindrome (e.g., 121, 55, 8) — the absolute difference becomes 0, and since 0 does not have a standard prime factorization, this must be reported directly as 0 rather than processed through the factor-counting loop.

1. **Reverse the digits:** Repeatedly extract the last digit of n using $n \% 10$, append it to a running value ($\text{reversed} = \text{reversed} \times 10 + \text{digit}$), and remove the last digit from n via integer division ($n /= 10$). Continue until n becomes 0.
2. **Compute the absolute difference:** Once the reversed number is built, compute $\text{diff} = |\text{original} - \text{reversed}|$. If diff equals 0 (a palindromic input), the answer is simply 0.
3. **Count the factors of 2:** While diff is evenly divisible by 2, divide it by 2 and increment a counter. Stop once diff becomes odd.
4. **Output the counter:** The final counter value is the exponent of 2 in diff 's prime factorization — this is the answer.

Java Solution:

```
import java.util.Scanner;

class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        long n = sc.nextLong();

        long original = Math.abs(n);
        long reversed = reverseDigits(original);
        long diff = Math.abs(original - reversed);

        int count = 0;
        while (diff != 0 && diff % 2 == 0) {
            diff /= 2;
            count++;
        }

        System.out.println(count);
    }

    private static long reverseDigits(long num) {
        long reversed = 0;
        while (num > 0) {
            reversed = reversed * 10 + (num % 10);
            num /= 10;
        }
        return reversed;
    }
}
```

Solution:

1. **Read the input:** `main()` reads a single integer `n` via Scanner, matching the Input Format.
2. **Reverse the digits:** `reverseDigits()` extracts the last digit of `num` with `num % 10`, appends it to a running total, and removes it from `num` using integer division, continuing until `num` is fully consumed.
3. **Compute the difference and handle palindromes:** `diff = |original - reversed|` is calculated once. If `diff` is 0, the while loop condition fails immediately, leaving count at 0 without any extra branching.
4. **Count and print the factors of 2:** The while (`diff % 2 == 0`) loop repeatedly halves `diff` and increments count for each successful division. Once `diff` becomes odd, the loop ends and count is printed — matching the Output Format.

Complexity Analysis:

Time Complexity: $O(\log n)$

Reversing the digits of `n` takes time proportional to its digit count, i.e., $O(\log_{10} n)$. Counting the factors of 2 in the difference takes time proportional to how many times it can be halved, at most $O(\log_2 \text{diff})$. Both stages are logarithmic, so the overall time complexity is $O(\log n)$.

Space Complexity: $O(1)$

The solution uses only a fixed number of scalar variables (`original`, `reversed`, `diff`, `count`) regardless of input size, so the space complexity is constant.